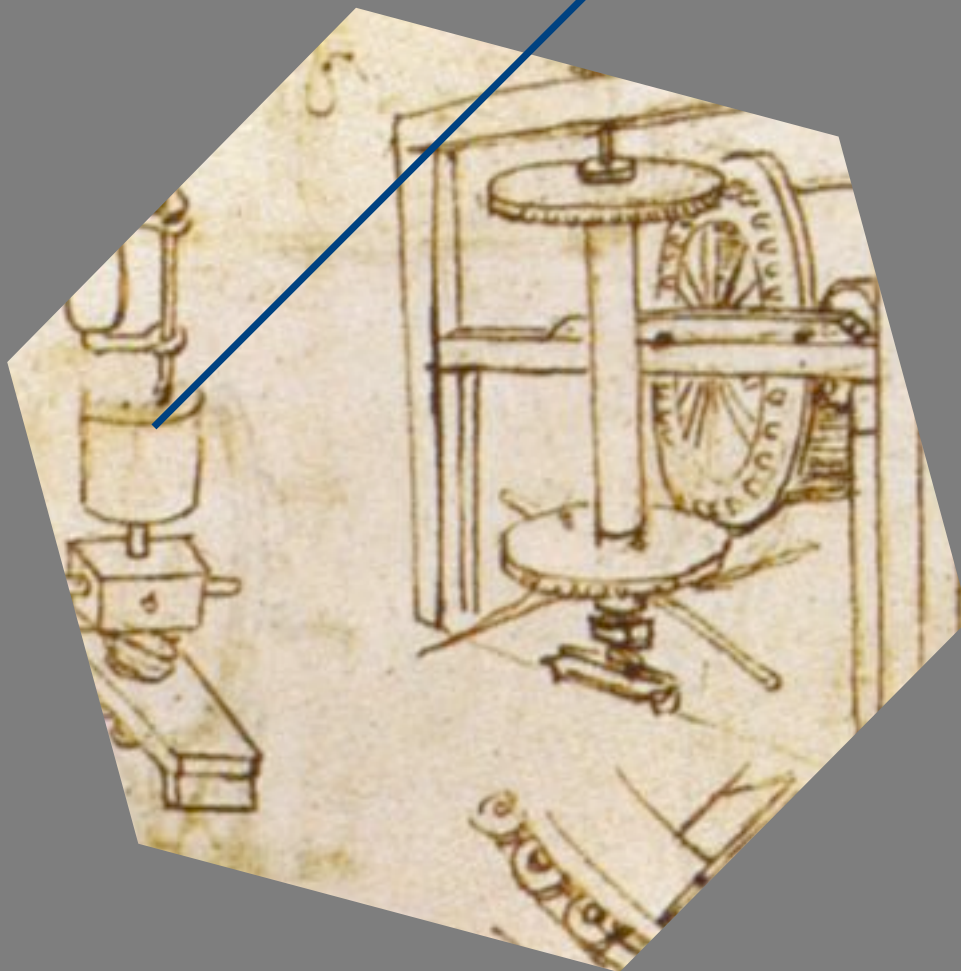


Web Engineering 2

Wintersemester 2026/27
Bachelor Medieninformatik

Prof. Dr. Jens von Pilgrim

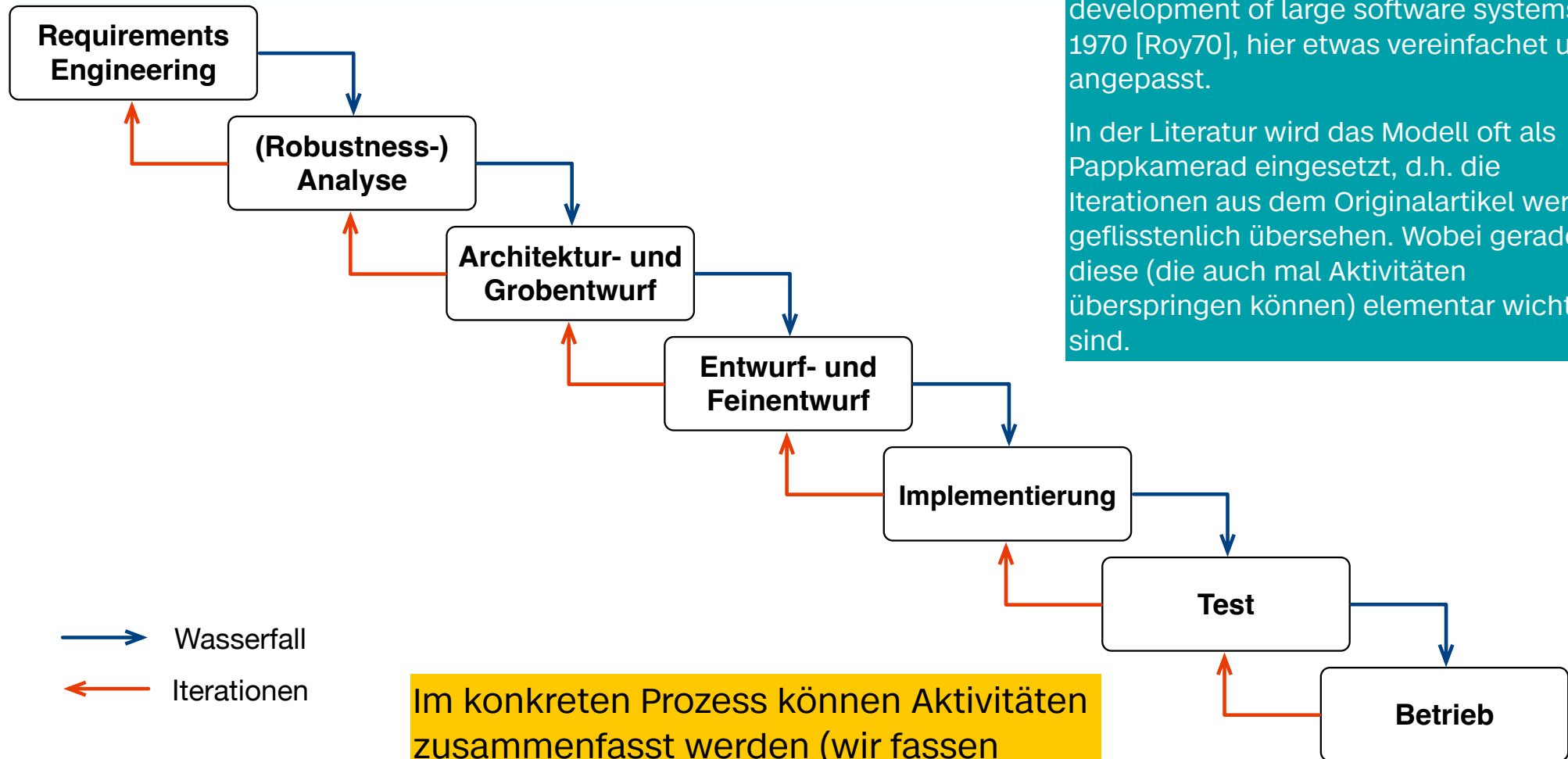
Stand: 15.9.2026



Überblick

Software Engineering in a Nutshell

Aktivitäten des Software Engineering



Wassefallmodell mit Iterationen nach Winston W. Royce. Managing the development of large software systems. 1970 [Roy70], hier etwas vereinfacht und angepasst.

In der Literatur wird das Modell oft als Pappkamerad eingesetzt, d.h. die Iterationen aus dem Originalartikel werden geflissentlich übersehen. Wobei gerade diese (die auch mal Aktivitäten überspringen können) elementar wichtig sind.

Im konkreten Prozess können Aktivitäten zusammenfasst werden (wir fassen Implementierung und Test zusammen).

Requirements Engineering

Wir erfassen (mit Kunden und Domänenexperten) die Anforderungen: Wir führen Interviews und Workshops durch. Dabei können wir schon Modelle erstellen, etwa

- **Domänenklassenmodelle:** Selbst mit Experten, die nicht nah an der Informatik sind, kann man einfache Klassendiagramme (ohne Methoden, nur einfache Typen, ohne Interfaces, ohne Vererbung) erstellen. Folgendes sollte erfasst werden:
 - Klassen mit Namen und Attributen (ggf. mit Typ und Constraints)
 - Assoziationen mit Rollennamen und Multiplizitäten (also statt Attributen)
- Aktivitätsmodelle oder State Machines je nach Bedarf

Weitere hilfreiche Methoden sind

- Wireframes
- Prototypen
- Rollenspiele

Analyse

In der Analyse erstellen wir formale Modelle (oft in Form von Diagrammen), also etwa

- **Systemkontext-Diagramm**

- Wir grenzen den Kontext ab, d.h. wir identifizieren, was unser System ausmacht und mit was es zusammenarbeitet — und was außerhalb unseres Kontextes ist und uns nicht interessiert.

- **Anwendungsfall-Diagramm**

- dabei sollten Vererbung, Include- und Extends-Beziehungen nur im Ausnahmefall eingesetzt werden. (Tipp: Preconditions verwenden!)
- mit Anwendungsfallbeschreibungen — ohne diese sind die Anwendungsfalldiagramme relativ sinnlos!
- und/oder **Aktivitäts-Diagrammen** — die man teilweise als Ergänzung oder statt textueller Ablaufbeschreibungen einsetzen kann.

- **Domänenklassen-Diagramm** (auch fachliches Klassenmodell genannt)

- Meist sollte dies bereits vorher mit den Domänenexperten erstellt worden sein — dann überarbeiten wir es nur. Ansonsten müssen wir es selbst erstellen und Rücksprache halten.
- Nur selten benötigen wir kein Domänenklassendiagramm — dies ist aber wirklich die Ausnahme!

- **State Machines**

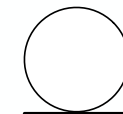
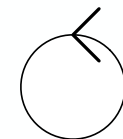
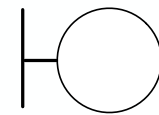
- sofern es sich anbietet. Bei manchen Projekten wird man dies vorher mit Domänenexperten zusammen erstellen (hier sind dann Rollenspiele oft hilfreich). Bei anderen Projekten erstellen wir es selbst und halten Rücksprache. Es gibt auch Projekte, wo State Machines nicht sinnvoll sind.
- Typische Beispiele:
 - Zustände einer Bestellung und/oder der enthaltenen Artikel (erfasst, reserviert, verpackt etc.)
 - Zustände von Reservierungen (Hotelbuchung, etc.)
 - Zustände von eingereichten Beiträgen

Robustness Analysis

Ein sehr hilfreicher Schritt, um die **Lücke zwischen Anforderungen und Entwurf zu schließen**, ist die von Ivar Jacobson [Jab92] entwickelte Robustness Analysis.

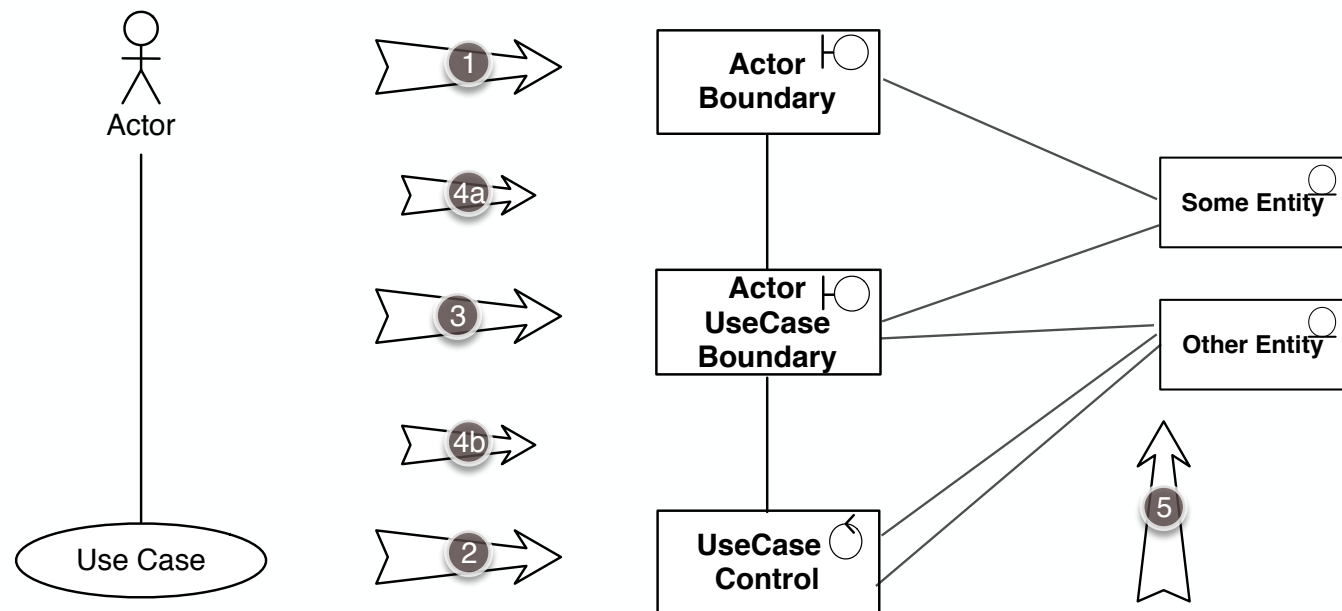
Hierbei leiten wir kanonisch (also regelgeleitet) Klassen aus den Anforderungsdokumenten (Use Case Model und Domain Model) ab. Dabei kommen drei **Stereotype** zum Einsatz:

- **Boundary Class** (Schnittstellenklasse), diese werden aus dem Use Case Model abgeleitet
- **Control Class** (Kontrollklasse), diese werden aus dem Use Case Model abgeleitet.
- **Entity Class** (Entitätsklasse), diese werden aus dem Domain Model abgeleitet



Ableitung der Entwurfsklassen aus den Anforderungsmodellen

Grundsätzlich kann man verschiedene Ableitungsregeln (Transformationen) anwenden. In [Jac92] werden aus (1) Akteuren (und (3) Assoziationen zu Akteuren) Boundary Classes, aus (2) Use Cases Control Classes. Assoziationen zwischen Control- und Entitätsklassen entsprechend Anwendungsfallbeschreibung. Zwischen den Klassen werden (4) Assoziationen entsprechend den Beziehungen im Ausgangsmodell erstellt. (5) Entitäten werden aus Domänenklassen übernommen und entsprechend den Anwendungsfällen anderen Klassen zugeordnet.



Wir werden die Regeln pragmatisch anwenden, d.h. bei Bedarf etwas abwandeln.

Entwurf

Der Entwurf kann unterteilt werden in:

- **Architekturauswahl und Grobentwurf**

- Für das gesamte System, ggf. auch für einzelne Komponenten, wird eine Architektur ausgewählt (wir werden REST und Schichtenarchitektur auswählen) und erhalten so Komponenten bzw. **Komponenten-Diagramme**.
- Wir verteilen die Klassen (aus der Robustness Analysis) auf die Komponenten

- **Feinentwurf**

- Wir wählen Frameworks und Bibliotheken aus und passen unsere Klassen diesen (und den eingesetzten Programmiersprachen) an. Hier erstellen wir vorwiegend **Klassendiagramme**.

Implementierung, Test, Betrieb

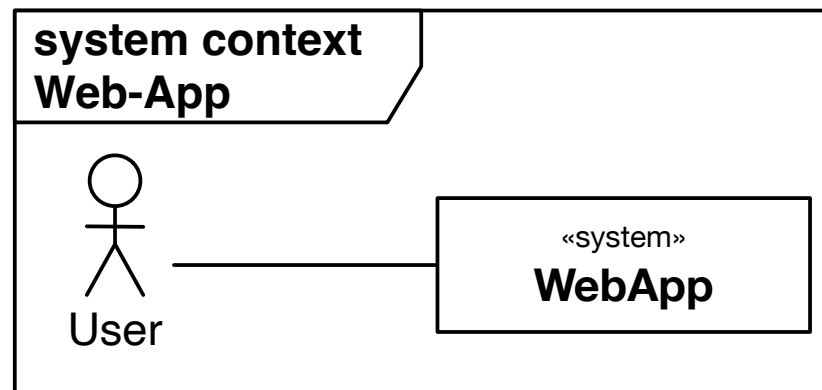
Mit modernen Techniken und Verfahren verschmelzen diese Aktivitäten immer mehr:

- Wir arbeiten **testgetrieben**, erstellen also Test und Implementierung zusammen.
- **Continuous Integration**: Nach dem Push und vor dem Merge einer Änderung werden alle Tests ausgeführt und, falls diese fehlschlagen, der Merge abgelehnt.
- **Continuous Deployment**:
 - Man setzt weitere Tests ein, u.a. automatisierte **Akzeptanztests** (bzw. sogenannte End-to-End-Tests)
 - falls diese erfolgreich sind, kann die neue Version automatisch deployed, also in Betrieb genommen werden (also etwa ein gebautes Docker-Image gestartet werden).

Unsere Web-App: Anforderungsanalyse

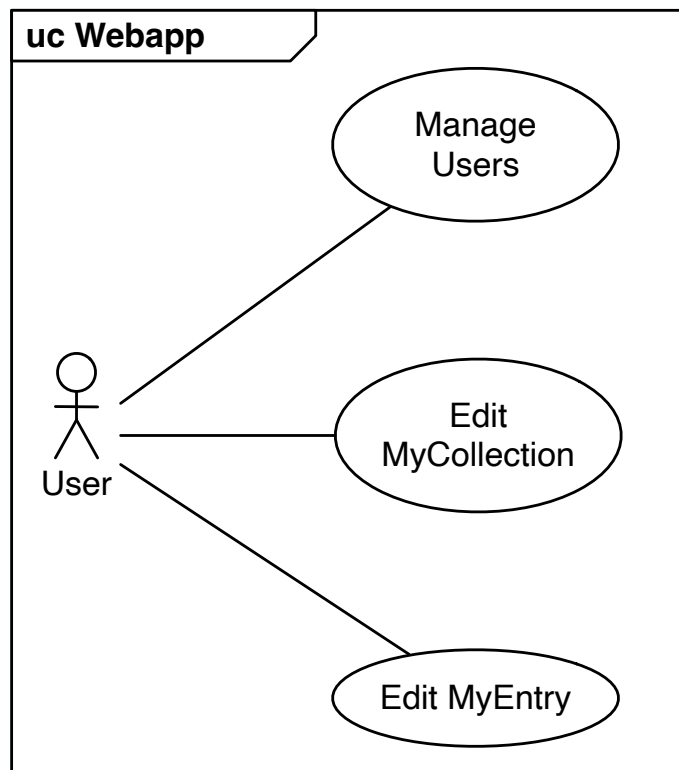
Systemkontext

Wir wollen eine WebApp entwickeln. Wir starten mit einem Systemkontext-Diagramm. Darin sehen wir das System und alle Akteure, die mit dem System interagieren.*)



Anwendungsfälle

Was können die Akteure machen? Dies definieren wir mit Anwendungsfällen.*)



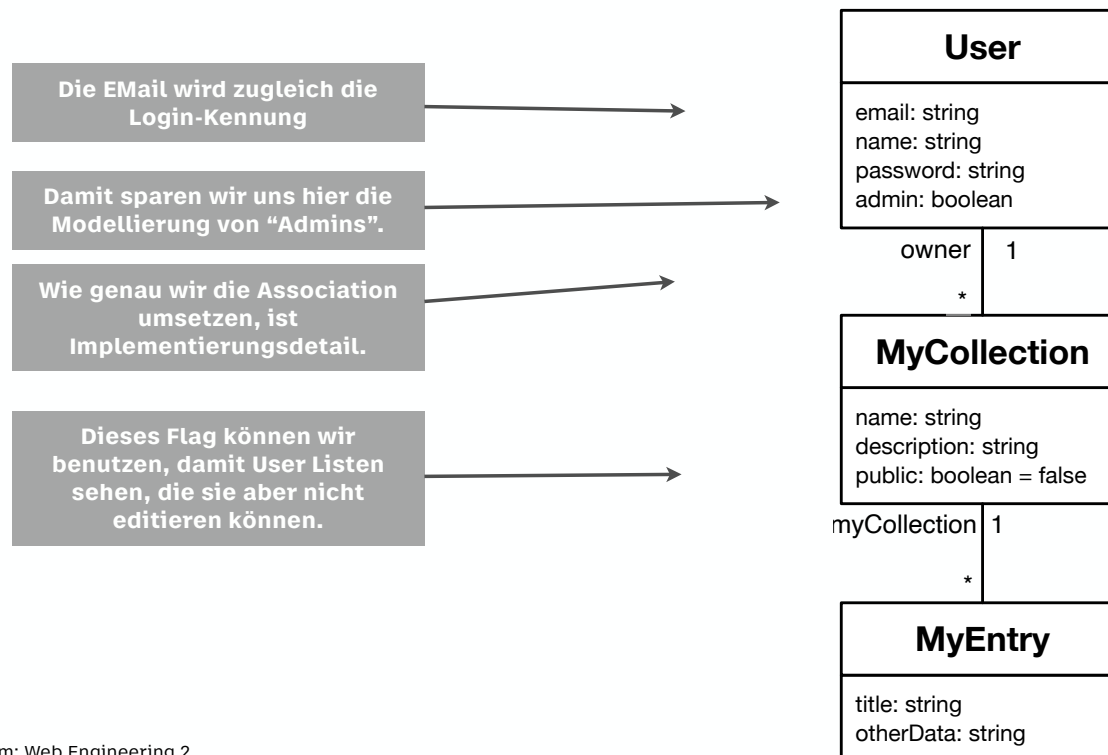
use case: Manage Users
pre: User ist angemeldet und Admin
flow: User muss neue User anlegen können
 ...

use case: Edit MyCollection
pre: User ist angemeldet
flow: User muss neue MyCollection anlegen können
 ...

use case: Edit MyEntry
pre: User ist angemeldet
flow: User muss neuen MyEntry anlegen können
 ...

Domänenklassen

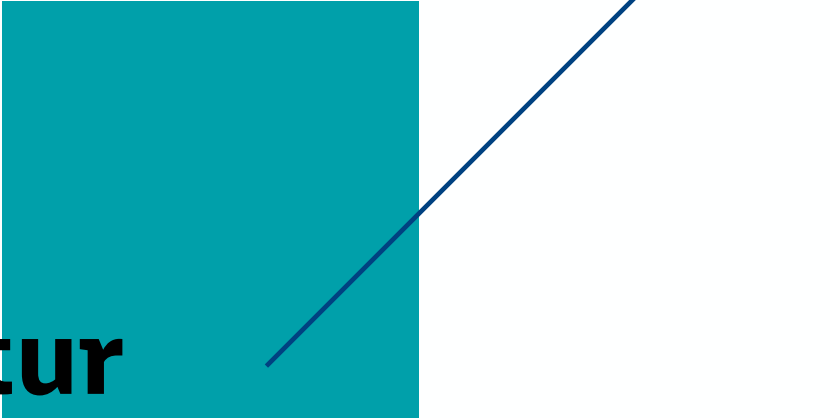
Welche Daten kommen in unserem System vor? Wir modellieren dies mit einem Domänenklassen-Modell, auch fachliches Klassenmodell genannt.



Dieses sehr allgemeine Schema passt für viele echte Anwendungen. Du kannst "MyCollection" und "MyEntry" einfach ersetzen durch

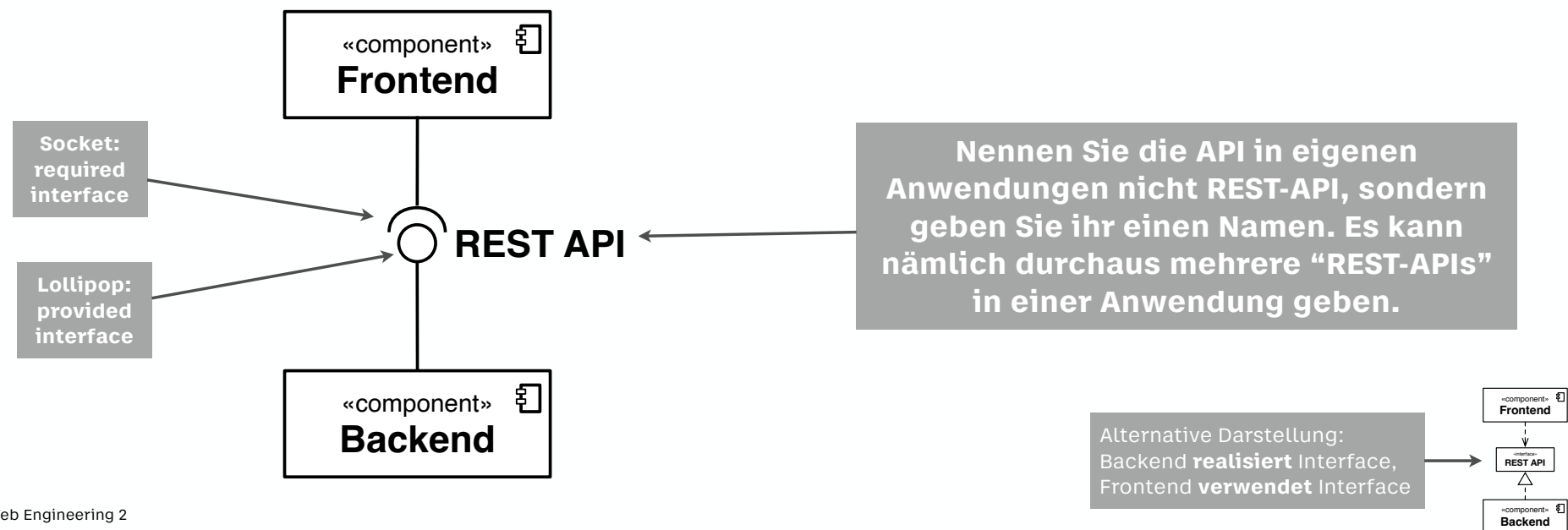
- Einkaufskorb und Artikel
- MessageThread und Message
- Kategorie und Blogeintrag
- Team und Spieler
- ...

Architektur



Architektur

Unser System ist eine klassische Client-Server-Anwendung. Der Client ist das Frontend, der Server das Backend. Beide werden über eine REST-API kommunizieren, wie es bei Web-Apps typisch ist.



Representational State Transfer: REST

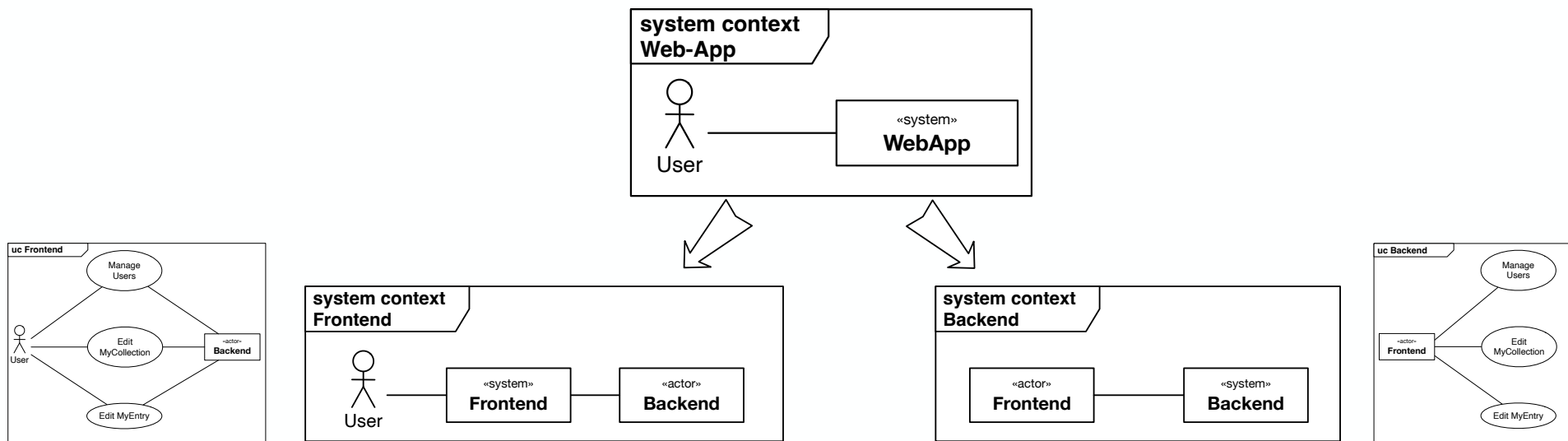
REST ist ein Architekturstil, entwickelt von Roy Fielding in seiner Dissertation [Fie00].

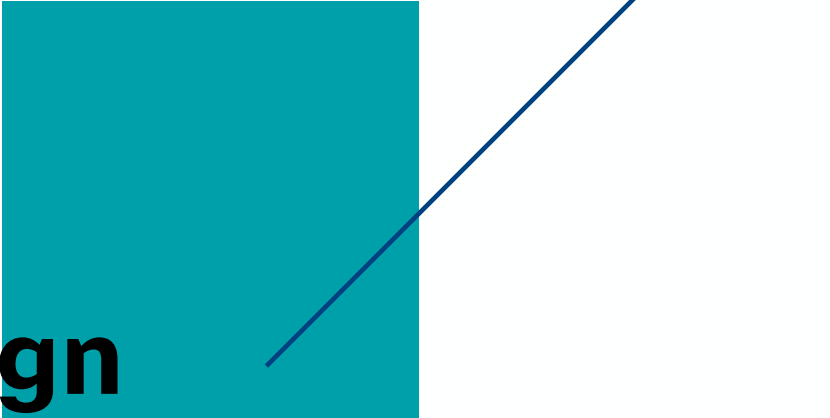
“The name “Representational State Transfer” is intended to evoke an image of how a well-designed Web application behaves: a network of **web pages (a virtual state-machine)**, where the user progresses through the application by selecting **links (state transitions)**, resulting in the next **page (representing the next state of the application)** being transferred to the user and rendered for their use.” [Fie00, S. 190, H.d.V.]



Eine Anwendung, zwei Systeme

Die strikte Trennung der Anwendung in Frontend und Backend führt dazu, dass wir beide Komponenten als eigenständige (Sub-) Systeme bzw. als eigenständige Entwicklungsprojekte begreifen können. Damit ergeben sich zwei Kontextdiagramme.*)

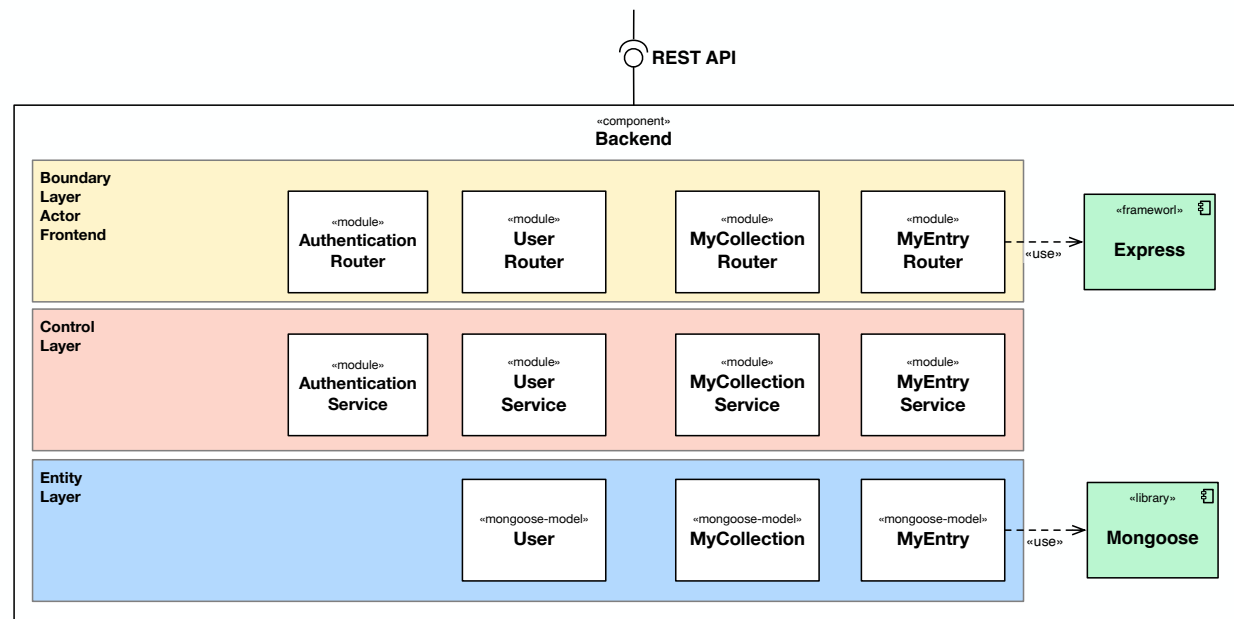




Design

Backend

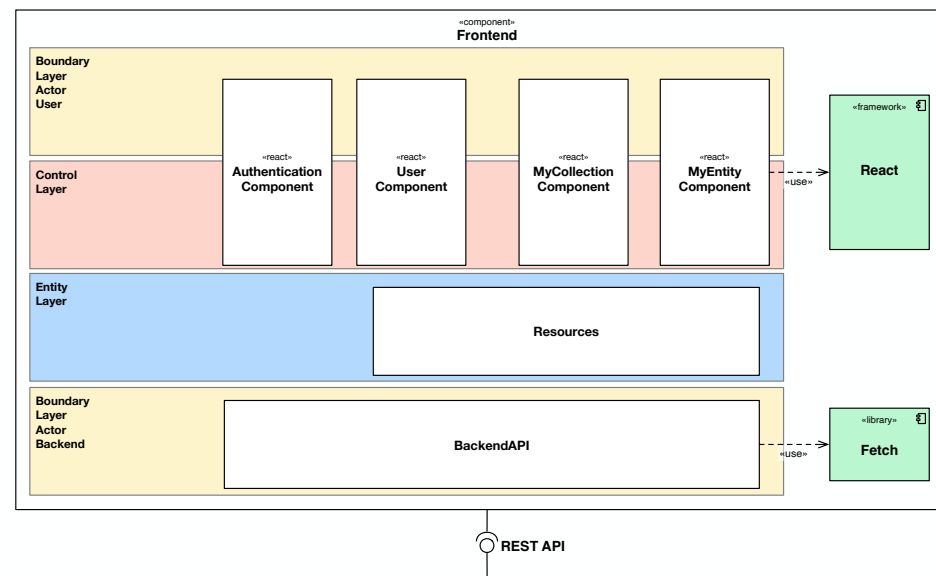
Für das Backend werden wir eine Schichtenarchitektur wählen. Für die Umsetzung der einzelnen Schichten werden spezifische Frameworks zum Einsatz kommen.



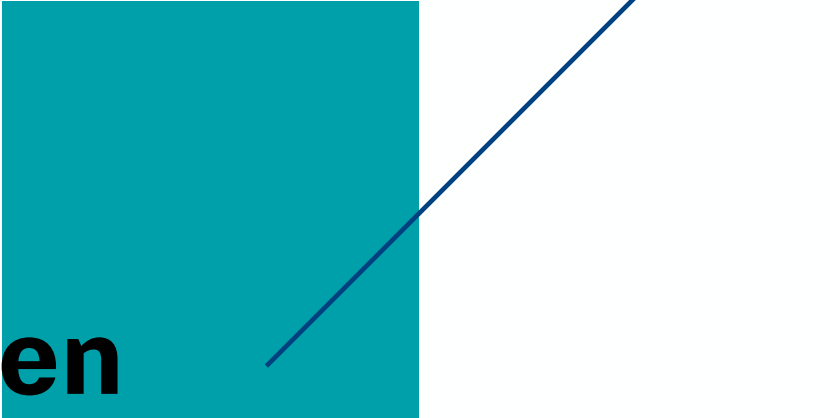
Frontend

Das Frontend werden wir mit **React** entwickeln. Dieses Framework bestimmt maßgeblich das (Software-) Design unseres Frontends. Im Zentrum stehen dabei sogenannte (React-) Components, aus denen wir die GUI zusammensetzen werden.

Zur Kommunikation mit dem Backend werden wir die (im Browser vorhandene) **Fetch**-API verwenden.



Technologien



JavaScript

Solide JavaScript-Kenntnisse werden vorausgesetzt!

Wir arbeiten *indirekt* mit JavaScript. JavaScript ist

- **funktional**, d.h. Funktionsreferenzen können Variablen zugewiesen werden
- **dynamisch typisiert**, d.h. Typinformationen liegen erst zu Laufzeit vor
- verwendet **prototypische Vererbung**
- ist **single-threaded**, durch **Event-Loop** ist asynchrone Programmierung möglich
- unterstützt seit ES 2015 **Klassen** und **Module**
- syntaktisch ähnlich zu Java oder C++ (Punktnotation)
- semantisch ähnlich zu Lisp (Scheme)



Empfohlene Literatur: Flanagan, David:
JavaScript – Das Handbuch für die Praxis

TypeScript

Solide TypeScript-Kenntnisse werden vorausgesetzt!

Wir arbeiten mit TypeScript. TypeScript ergänzt JavaScript um ein statisches Typsystem (und ein paar andere Dinge).

- Typannotationen können (bzw. müssen) im Quelltext angegeben werden.
- Hierzu wird die Colon-Notation verwendet.
- Der Compiler führt eine statische Typprüfung durch.
- Mittels Inferenz sind die Typannotationen an vielen Stellen unnötig.
- Zur Typisierung können Interfaces definiert werden.
- Es gibt spezielle Typkonstrukte, um JavaScript-Eigenschaften abzubilden, u.a. Union- und Intersectiontypes



Empfohlene Literatur: Cherny, Boris:
Programmieren in TypeScript

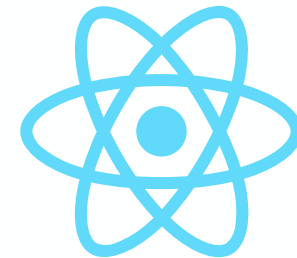
MERN

Wir werden den sogenannten MERN-Stack einsetzen.
MERN steht für

- **MongoDB:** Dokumenten-basierte Datenbank
- **Express:** Framework zum Mappen von URIs auf Funktionen
- **React:** High-Level Web-UI-Framework
- **Node.js:** Laufzeitumgebung für JavaScript



express



Es gibt ein paar solche “Stacks”, wie LAMP (Linux, Apache, MySQL, PHP), MEAN (MongoDB, Express, Angular, Node.js) — und natürlich noch viel mehr ohne coole Akronym. Und zu diesen Tools kommen immer noch weitere hinzu.

Literatur

- [Che19] Cherny, Boris: Programmieren in TypeScript. O'Reilly, 2019
- [Fla21] Flanagan, David: JavaScript – Das Handbuch für die Praxis. 7. Auflage. O'Reilly, 2021. ISBN 9781098129125
- [Jac92] Jacobson, Ivar: Object-Oriented Software Engineering: A Use Case Driven Approach. Addison- Wesley Professional, 1992 (acm press)
- [OMG17] OMG: OMG Unified Modeling Language (OMG UML), Version 2.5.1. formal/17-12-05. Needham, MA: Object Management Group, Dezember 2017. <https://www.omg.org/spec/UML/2.5.1/>
- [PR15] Pohl, Klaus; Rupp, Chris: Basiswissen Requirements Engineering. 4. Auflage. dpunkt.verlag, 2015. ISBN 978-3-86491-673-1
- [RQ12] Rupp, Chris; Queins, Stefan: UML 2 glasklar. Praxiswissen für die UML-Modellierung und Zertifizierung. 4., überarb. u. erw. Aufl. Hanser Fachbuchverlag, 2012. DOI [10.3139/9783446431973](https://doi.org/10.3139/9783446431973). ISBN 978-3-446-43057-0
- [RS99] Rosenberg, Doug; Scott, Kendall: Use Case Driven Object Modeling with UML. A Practical Approach. Addison-Wesley, 1999 (Object Technology Series). ISBN 0201432897
- [SKRG20] Starke, Gernot; Koschel, Arne; Rausch, Andreas; Gharbi, Mahbouba: Basiswissen für Softwarearchitekten. 4. Auflage. dpunkt.verlag, 2020. ISBN 9783969100127
- [Win05] Winter, Mario: Methodische objektorientierte Softwareentwicklung. Eine Integration klassischer und moderner Entwicklungskonzepte. Heidelberg : dpunkt.verlag, 2005. ISBN 3-89864-273-9
- [Zör22] Zörner, Stefan: Software-Architekturen dokumentieren und kommunizieren. 3. Auflage. Hanser, 2022. DOI [10.3139/9783446472464](https://doi.org/10.3139/9783446472464)